

## **PROJETO FINAL DE SISTEMAS OPERACIONAIS - RELATÓRIO**

Sistemas Operacionais – Tema 3

*Prof. Marcelo Zanchetta do Nascimento*

### **Alunos(as)**

Arthur Fernandes de Sousa - 12411BCC097

Erik Alves Scagliarini - 12411BCC060

Gustavo Ferreira Maia - 12411BCC094

João Pedro de Moraes e Menezes - 12421BCC087

Ricardo Garcia Fernandes Vezzani - 12411BCC052

**Uberlândia/2026**

## **1. Introdução**

Este relatório detalha a implementação do algoritmo de Escalonamento por Prioridade (Priority Scheduling) no sistema operacional XV6, conforme as diretrizes do projeto da disciplina de Sistemas Operacionais (GBC045) da Universidade Federal de Uberlândia [1]. O objetivo principal foi modificar o kernel do XV6 para suportar a definição de prioridades para processos e garantir que o escalonador selecione o processo com a maior prioridade (menor valor numérico) para execução. O trabalho envolveu a criação de uma nova chamada de sistema (`setpriority`), alterações na estrutura de dados de processo e na lógica do escalonador, além do desenvolvimento de programas de usuário para testar a funcionalidade implementada.

## **2. Requisitos do Projeto (Etapa 2)**

A Etapa 2 do projeto focou na implementação prática de um algoritmo de escalonamento no XV6. Para o Tema 3, "Priority Scheduling", os requisitos específicos incluíam:

- **Criação e Modificação de Chamadas de Sistema:** Implementar a syscall `setpriority(int)` para permitir que processos de usuário definam sua própria prioridade.
- **Alterações no Kernel:** Modificar a estrutura de processo (`struct proc`) para incluir um campo de prioridade e ajustar a função `scheduler()` para utilizar essa prioridade na seleção do próximo processo a ser executado.
- **Desenvolvimento de Programa de Usuário para Testes:** Criar programas que demonstrem o funcionamento do escalonador por prioridade.
- **Implementação/Adaptação do Algoritmo de Escalonamento:** Substituir ou adaptar o escalonador Round Robin padrão do XV6 pelo Priority Scheduling.
- **Experimentos e Análise Comparativa:** Realizar experimentos para demonstrar o comportamento do algoritmo e compará-lo com o escalonador Round Robin original.

do XV6. O relatório deve apresentar os testes realizados, métricas coletadas, resultados experimentais e uma análise crítica da solução.

### 3. Implementação do Escalonamento por Prioridade no XV6

#### 3.1. Modificações na Estrutura de Dados (proc.h)

Para suportar o escalonamento por prioridade, foi necessário adicionar um novo campo à estrutura struct proc, que representa um processo no XV6. No arquivo proc.h, a seguinte linha foi incluída:

```
int priority;           // Variável criada; Process priority
```

Este campo priority armazena um valor inteiro que indica a prioridade do processo. Por convenção, valores menores representam prioridades mais altas, o que significa que um processo com priority = 1 será executado antes de um processo com priority = 10.

#### 3.2. Inicialização da Prioridade (proc.c)

Ao criar um novo processo, é fundamental que sua prioridade seja inicializada. No arquivo proc.c, dentro da função allocproc(), que é responsável por alocar e inicializar uma nova estrutura de processo, a prioridade padrão foi definida como 10:

```
p->priority = 10;      // Inicializa a prioridade do processo
```

Essa inicialização garante que todos os processos recém-criados tenham uma prioridade base, que pode ser posteriormente alterada por meio da syscall setpriority().

#### 3.3. Algoritmo do Escalonador (proc.c)

O coração da implementação do Priority Scheduling reside na modificação da função scheduler() no arquivo proc.c. Esta função é executada por cada CPU para selecionar o próximo processo a ser executado. A lógica original do Round Robin foi substituída por um mecanismo que busca o processo RUNNABLE com a maior prioridade (menor valor numérico).

O novo algoritmo de escalonamento funciona da seguinte forma:

- 1 O escalonador itera sobre todos os processos na tabela de processos (proc[]).
- 2 Para cada processo, ele verifica se o estado é RUNNABLE (pronto para execução).
- 3 Entre os processos RUNNABLE, ele mantém um ponteiro para o processo com a menor prioridade (best).
- 4 Se um processo RUNNABLE com prioridade menor que a do best atual for encontrado, ele se torna o novo best.
- 5 Após varrer todos os processos, se um best processo for encontrado, sua prioridade é a mais alta entre os processos RUNNABLE.
- 6 O escalonador então realiza a troca de contexto (swtch) para o processo best, permitindo que ele execute.

Este mecanismo garante que processos com prioridades mais altas sejam preferidos para execução, implementando efetivamente o escalonamento por prioridade.

#### 4. Chamada de Sistema setpriority()

##### 4.1. Definição e Interface

Para permitir que processos de usuário modifiquem suas próprias prioridades, uma nova chamada de sistema, setpriority(int), foi criada. A integração dessa syscall no XV6 envolveu várias etapas:

- **syscall.h**: Um novo número de syscall foi definido para setpriority:

```
#define SYS_setpriority 22
```

- **defs.h**: O protótipo da função kernel que implementa a syscall foi declarado:

```
uint64 sys_setpriority(void);
```

- **user.h**: A interface para programas de usuário foi adicionada, permitindo que eles chamem setpriority():

```
int setpriority(int);
```

- **usys.pl**: O script Perl que gera os stubs de syscall para o espaço de usuário foi atualizado para incluir setpriority:

```
entry("setpriority");
```

#### 4.2. Implementação no Kernel (sysproc.c e syscall.c)

A lógica da syscall setpriority() foi implementada no arquivo sysproc.c:

```
sys_setpriority(void)
{
    int priority;
    argint(0, &priority); // Obtém o argumento da syscall
    myproc()->priority = priority; // Atualiza a prioridade do processo atual
    return 0;
}
```

Esta função obtém o argumento inteiro passado pelo usuário (a nova prioridade) e atualiza o campo priority do processo atual (myproc()).

Finalmente, no arquivo syscall.c, a função sys\_setpriority foi registrada no vetor syscalls[], mapeando o número da syscall (SYS\_setpriority) para sua implementação no kernel:

```
[SYS_setpriority] sys_setpriority,
```

#### 5. Programas de Teste e Metodologia Experimental

Dois programas de usuário foram desenvolvidos para testar a funcionalidade do escalonamento por prioridade:

- **priotest.c**: Este é um teste básico que verifica se a syscall setpriority() funciona corretamente. Ele simplesmente chama setpriority(3) e imprime mensagens antes e depois da chamada para confirmar a alteração da prioridade do processo atual. Embora

útil para uma verificação inicial, não demonstra o comportamento do escalonador em um cenário com múltiplos processos concorrentes.

- **teste.c**: Este programa foi projetado para demonstrar o comportamento do escalonador por prioridade em um ambiente com múltiplos processos. Ele cria três processos filhos (nomeados "A", "B" e "C") usando `fork()`. Cada processo filho executa uma função `work()` que define sua própria prioridade e então imprime uma mensagem indicando que executou. As prioridades são definidas da seguinte forma:
  - Processo A: Prioridade 10
  - Processo B: Prioridade 5
  - Processo C: Prioridade 1
- **teste\_grafs.c**: Programa projetado para coleta dos dados para realização dos gráficos a respeito do Priority Scheduling. Ela nos permite calcular o **Turnaround Time** (tempo total de vida do processo) e o **Waiting Time** (estimado como o tempo total menos o tempo de execução).
- **grafs\_rr.c**: Programa com a mesma finalidade do 'teste\_grafs.c' porém para o algoritmo de Round Robin.

De acordo com a lógica do Priority Scheduling implementada (menor valor = maior prioridade), espera-se que o Processo C execute primeiro, seguido pelo Processo B e, por fim, pelo Processo A. O programa pai aguarda a finalização de todos os filhos usando `wait()`.

Todos os programas de teste (`priotest` e `teste`) foram adicionados ao `Makefile` do XV6, garantindo que sejam compilados e incluídos na imagem do sistema (`fs.img`), tornando-os acessíveis para execução no ambiente XV6.

## 6. Resultados Experimentais e Análise Crítica

### 6.1. Resultados Esperados

Com base na implementação do escalonador por prioridade e no programa `teste.c`, os resultados esperados são:

- O **Processo C** (prioridade 1) deve ser o primeiro a imprimir sua mensagem, indicando que foi o primeiro processo RUNNABLE a ser selecionado e executado pelo escalonador.
- Em seguida, o **Processo B** (prioridade 5) deve imprimir sua mensagem.
- Por último, o **Processo A** (prioridade 10) deve imprimir sua mensagem.

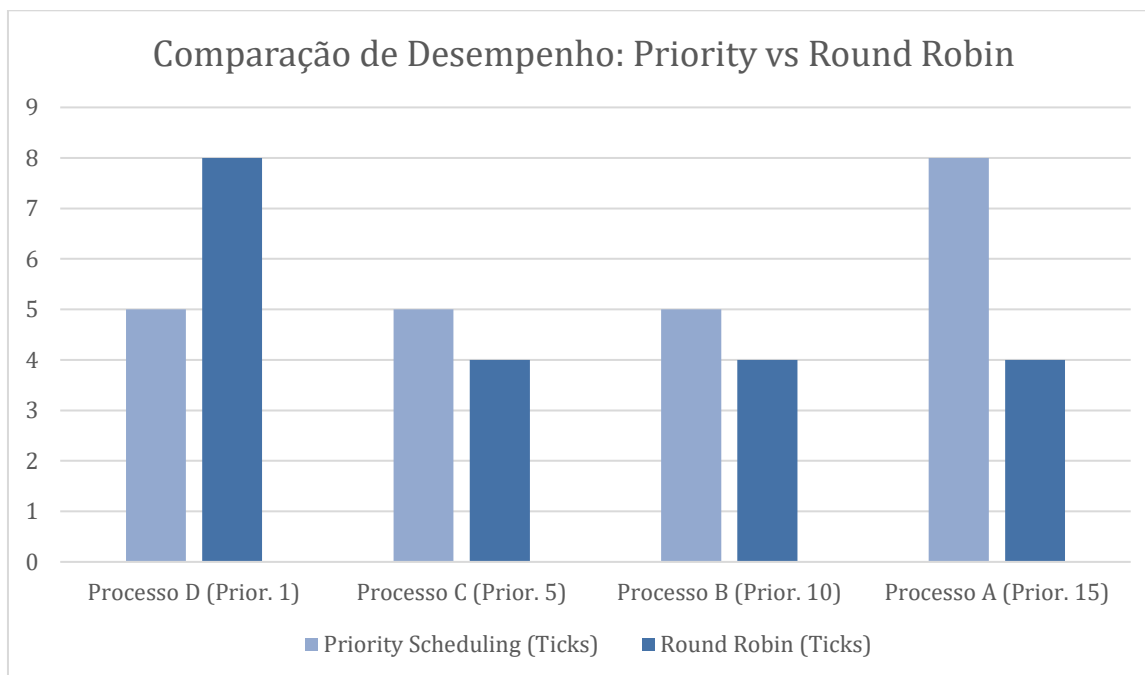
Essa sequência de execução demonstra que o escalonador está, de fato, priorizando processos com valores de prioridade menores, conforme o design do algoritmo.

## 6.2 Resultados Experimentais

Como dito na sessão 5, utilizamos dois arquivos para obtermos métricas de comparações ('teste\_grafs.c' e 'grafs\_rr.c'), onde foi adicionado um processo a mais para melhores fins comparativos. Os resultados obtidos foram:

| Algoritmo           | Processo | Prioridade | Turnaround (Ticks) |
|---------------------|----------|------------|--------------------|
| Priority Scheduling | D        | 1          | 5                  |
|                     | C        | 5          | 5                  |
|                     | B        | 10         | 5                  |
|                     | A        | 15         | 8                  |
| Round-Robin         | A,B, C   | -          | 4                  |
|                     | D        | -          | 8                  |

### 6.3. Análise Gráfica



### 6.4. Análise Crítica da Solução

#### Vantagens:

- **Controle sobre a Execução:** O Priority Scheduling permite que processos críticos ou sensíveis ao tempo recebam mais tempo de CPU, melhorando a responsividade do sistema para tarefas importantes. Porém como vimos a CPU acaba sendo monopolizada por processos com maiores prioridades. Já no Round-Robin a CPU é fatiada igualmente, o que nem sempre pode ser benéfico para o sistema.
- **Simplicidade de Implementação:** A adição de um campo de prioridade e a modificação da lógica de seleção no `scheduler()` são relativamente diretas, tornando o algoritmo fácil de entender e implementar em um kernel simples como o XV6.

#### Desvantagens e Considerações:

- **Starvation (Inanição):** A implementação atual do Priority Scheduling no XV6 não inclui mecanismos para prevenir a inanição. Um processo com baixa prioridade pode nunca ser executado se houver um fluxo constante de processos de alta prioridade. Em



sistemas reais, técnicas como *aging* (envelhecimento) são usadas para aumentar gradualmente a prioridade de processos que esperam por muito tempo.

- **Overhead de Busca:** A cada ciclo de escalonamento, o kernel precisa varrer a lista completa de processos RUNNABLE para encontrar o de maior prioridade. Embora para um número pequeno de processos no XV6 isso seja aceitável, em sistemas com muitos processos, essa busca linear pode introduzir um overhead significativo. Estruturas de dados mais eficientes (como filas de prioridade) poderiam mitigar isso.
- **Ausência de Preempção:** A implementação não detalha explicitamente a preempção baseada em prioridade. Se um processo de baixa prioridade estiver em execução e um processo de alta prioridade se tornar RUNNABLE, o escalonador atual só o selecionará na próxima vez que o processo em execução ceder a CPU ou for interrompido por um *timer interrupt*. Um escalonador preemptivo por prioridade interromperia imediatamente o processo de baixa prioridade.
- **Prioridades Estáticas:** As prioridades são definidas estaticamente pelos processos (ou pelo pai) e não mudam dinamicamente com base no comportamento do processo (e.g., uso de I/O vs. CPU-bound). Prioridades dinâmicas podem levar a um uso mais eficiente da CPU.

Comparado ao escalonador **Round Robin original do XV6**, que oferece uma distribuição de tempo de CPU mais justa entre todos os processos (evitando inanição por design), o Priority Scheduling introduz a capacidade de diferenciar a importância dos processos. No entanto, essa capacidade vem com o custo potencial de inanição para processos de baixa prioridade, um problema que o Round Robin não possui. A escolha entre os dois depende dos requisitos do sistema: Round Robin para justiça e Priority Scheduling para responsividade e atendimento a SLAs (Service Level Agreements) para tarefas críticas.

## 7. Conclusão

A implementação do Escalonamento por Prioridade no XV6 foi realizada com sucesso, demonstrando a capacidade de modificar o kernel para alterar o comportamento fundamental do sistema operacional. A criação da syscall setpriority() e a adaptação do scheduler()

permitiram que processos de usuário influenciem diretamente sua ordem de execução. Embora a solução atenda aos requisitos básicos do projeto, a análise crítica revela a necessidade de considerar mecanismos adicionais, como o *aging*, para mitigar problemas como a inanição em um ambiente de produção. Este trabalho proporcionou uma compreensão aprofundada dos mecanismos de escalonamento de processos e da arquitetura do kernel do XV6.

## Referências

- COX, Russ; MORRIS, Robert; KAASHOEK, Frans; et al. *xv6: a simple, Unix-like teaching operating system*. Cambridge: Massachusetts Institute of Technology (MIT), 2022. Disponível em: <https://pdos.csail.mit.edu/6.S081/2022/xv6/book-riscv-rev3.pdf>. Acesso em: jul. 2026.
- Silberschatz, A.; Galvin, P. B.; Gagne, G. *Operating System Concepts*, 10ª ed. Wiley. Acesso em: jul. 2026.
- **YOUTUBE**. *Playlist: The xv6 Kernel*. Disponível em: [https://www.youtube.com/watch?v=fWUJKH0RNFE&list=PLbtzT1TYeoMhTPzyTZboW\\_j7TPAnjv9XB](https://www.youtube.com/watch?v=fWUJKH0RNFE&list=PLbtzT1TYeoMhTPzyTZboW_j7TPAnjv9XB). Acesso em: jul. 2026.